

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ГЕОДЕЗИИ И
КАРТОГРАФИИ»
(ММИГАиК)
Факультет геоинформатики и информационной безопасности
(ФГиИБ)

ПРАКТИЧЕСКАЯ РАБОТА №2
по дисциплине: Методы машинного обучения в задачах обработки
пространственных данных

Работу выполнил:
Студент 2025-ФГиИБ-ПИ-1м
Сабитов Р.Р.
Преподаватель: Шириникин С.Г.

Москва, 2026 г.

Содержание

Введение.....	3
Методология	4
Результаты.....	7
Заключение	10
Список использованных источников	11

Введение

Prompt Engineering — это дисциплина, изучающая методы формулировки входных инструкций (пром프트в) для управления поведением больших языковых моделей (LLM) без изменения весов модели. Исследования показывают, что при неизменной модели разные формулировки одного и того же запроса могут приводить к качественно различным результатам как по содержанию, так и по структуре ответа. Данная работа представляет собой практический эксперимент, целью которого являлось эмпирическое сравнение шести ключевых техник на единой прикладной задаче в условиях локального запуска модели.

Актуальность эксперимента определяется возросшим интересом к развёртыванию LLM в закрытых контурах (on-premise) без передачи данных во внешние облачные сервисы. Платформа Ollama предоставляет унифицированный REST API и Python-интерфейс для локального запуска открытых моделей, что делает её удобным инструментом для воспроизводимых экспериментов.

Методология

В качестве тестового задания была выбрана задача классификации тональности отзыва клиента (Sentiment Analysis) — типичная задача обработки естественного языка с практическим применением в системах анализа обратной связи. Входной текст оставался неизменным для всех шести техник:

«Доставили быстро, но упаковка была помята, и одна деталь оказалась сломана.»

Отзыв намеренно выбран смешанным по тональности: содержит положительный аспект (скорость доставки) и явные негативные факты (физический ущерб товару), что формирует нетривиальную задачу для классификации и позволяет выявить различия в логике рассуждений между техниками.

В эксперименте использовались следующие техники:

1. Zero-Shot Prompting — прямой запрос без примеров и дополнительного контекста. Модель опирается исключительно на знания, полученные в процессе предобучения.
2. Few-Shot Prompting — запрос с несколькими демонстрационными примерами (in-context learning), иллюстрирующими желаемый формат и логику вывода.
3. Chain-of-Thought (CoT) — явное требование пошагового рассуждения (step-by-step reasoning) перед формулировкой итоговой метки. Метод введён Wei et al. (2022).
4. Role Prompting — назначение модели конкретной роли или персоны с целью управления стилем, глубиной и структурой ответа.
5. Self-Refine — двухэтапный итеративный процесс: первичный ответ с последующей самокритикой модели и генерацией улучшенной версии.
6. Template Pattern — принудительное заполнение заранее заданного структурированного шаблона с фиксированными полями.

Все запросы выполнялись последовательно через Python-библиотеку ollama с использованием метода ollama.chat(). Для каждой техники фиксировалось время генерации ответа. Модель gemma3:1b запусклась локально через платформу Ollama (endpoint http://localhost:11434) без подключения к внешним API.

```
import ollama
import time
import json

MODEL = "gemma3:1b"
REVIEW = "Доставили быстро, но упаковка была помята, и одна деталь оказалась сломана."

def run_prompt(name, messages):
    start = time.time()
    response = ollama.chat(model=MODEL, messages=messages)
    elapsed = round(time.time() - start, 2)
    content = response["message"]["content"]
    print(f"[{name}] ({elapsed} c)\n{content}\n")
    return {"response": content, "time_sec": elapsed}
```

Листинг 1 — Инициализация клиента и утилита замера

```
# Zero-Shot
run_prompt("Zero-Shot", [{
    "role": "user",
    "content": (
        "Классифицируй отзыв как ПОЗИТИВНЫЙ, НЕГАТИВНЫЙ или НЕЙТРАЛЬНЫЙ "
        "и объясни почему.\n\n"
        f"Отзыв: «{REVIEW}»"
    )
}])

# Few-Shot
run_prompt("Few-Shot", [{
    "role": "user",
    "content": (
        "Классифицируй отзывы. Примеры:\n\n"
        "Отзыв: «Всё отлично, очень доволен покупкой!»\n"
        "Метка: ПОЗИТИВНЫЙ — клиент выражает явное удовлетворение.\n\n"
        "Отзыв: «Товар сломан, деньги вернуть невозможно.»\n"
        "Метка: НЕГАТИВНЫЙ — явный негатив по нескольким аспектам.\n\n"
        "Отзыв: «Получил посылку вовремя.»\n"
        "Метка: НЕЙТРАЛЬНЫЙ — констатация факта без оценки.\n\n"
        f"Отзыв: «{REVIEW}»\nМетка:"
    )
}])
```

Листинг 2 — Zero-Shot и Few-Shot промпты

```
run_prompt("Chain-of-Thought", [{
    "role": "user",
    "content": (
        "Классифицируй отзыв как ПОЗИТИВНЫЙ, НЕГАТИВНЫЙ или НЕЙТРАЛЬНЫЙ.\n"
        "Сначала пошагово разбери каждый аспект, "
```

```

        "затем вынеси итоговую метку.\n\n"
        f"Отзыв: «{REVIEW}»\n\nРассуждение:"
    )
})

```

Листинг 3 — Явное пошаговое рассуждение

```

run_prompt("Role Prompting", [
    {
        "role": "system",
        "content": (
            "Ты — старший аналитик службы контроля качества маркетплейса "
            "с 10-летним опытом. Структурируй ответ строго по пунктам: "
            "1) Метка, 2) Позитивные аспекты, "
            "3) Негативные аспекты, 4) Рекомендация продавцу."
        )
    },
    {
        "role": "user",
        "content": f"Классифицируй и проанализируй отзыв: «{REVIEW}»"
    }
])

```

Листинг 4 — Системная роль эксперта

```

messages = [{"role": "user",
              "content": f"Классифицируй отзыв: «{REVIEW}»"}]

r1 = ollama.chat(model=MODEL, messages=messages)
messages.append({"role": "assistant", "content": r1["message"]["content"]})
messages.append({
    "role": "user",
    "content": (
        "Критически оцени свой ответ: что можно улучшить или уточнить? "
        "Дай финальную, исправленную и более точную версию."
    )
})
r2 = ollama.chat(model=MODEL, messages=messages)

```

Листинг 5 — Итеративное самоулучшение ответа

```

run_prompt("Template Pattern", [{
    "role": "user",
    "content": (
        f"Проанализируй отзыв и заполни шаблон строго в указанном формате:\n\n"
        f"Отзыв: «{REVIEW}»\n\n"
        "- Метка: [ПОЗИТИВНЫЙ / НЕГАТИВНЫЙ / НЕЙТРАЛЬНЫЙ / СМЕШАННЫЙ]\n"
        "- Позитивные аспекты: [перечисли через запятую]\n"
        "- Негативные аспекты: [перечисли через запятую]\n"
        "- Уверенность: [Низкая / Средняя / Высокая]\n"
        "- Рекомендация для продавца: [1 предложение]"
    )
}])

```

Листинг 6 — Структурированный шаблон вывода

Результаты

Техника	Итоговая метка	Время (с)	Объём ответа	Следование формату
Zero-Shot	НЕЙТРАЛЬНЫЙ	17,59	Средний	Свободный
Few-Shot	НЕГАТИВНЫЙ	4,15	Минимальный	Строгий
Chain-of-Thought	НЕГАТИВНЫЙ	16,07	Средний	Пошаговый
Role Prompting	СМЕШАННЫЙ	55,86	Максимальный	Структурированный
Self-Refine	НЕГАТИВНЫЙ	52,81	Большой	Итеративный
Template Pattern	НЕГАТИВНЫЙ	12,08	Малый	Точный

Таблица 1 – Результаты проведения экспериментов

Zero-Shot. Модель корректно идентифицировала присутствие разнородных аспектов, однако не произвела их взвешивание по значимости и присвоила метку НЕЙТРАЛЬНЫЙ. Сломанная деталь (критический дефект товара) была приравнена по весу к скорости доставки, что привело к неверной итоговой классификации. Данное поведение согласуется с известным ограничением Zero-Shot на задачах с неоднозначной тональностью: без контекста модель склонна к усреднению.

Few-Shot. Демонстрация трёх прецедентов сформировала у модели чёткую классификационную логику: наличие жалобы на физический ущерб товару безусловно соответствует метке НЕГАТИВНЫЙ. Результат получен за рекордные 4,15 секунды при полной корректности классификации. Техника подтверждает свою эффективность как метод in-context learning для задач с чётко определённым форматом вывода.

Chain-of-Thought. Модель явно перечислила положительные аспекты («доставили быстро»), отрицательные аспекты («упаковка помята», «деталь сломана») и обоснованно заключила, что негативы перевешивают позитив. CoT обеспечивает наиболее прозрачную логику рассуждения и наилучшую аудируемость решения из всех протестированных техник.

Role Prompting. Назначение роли эксперта-аналитика радикально изменило характер ответа: модель сгенерировала семь конкретных рекомендаций продавцу, подробный разбор каждого аспекта и

дополнительные предложения по улучшению логистики. Время генерации составило 55,86 секунды. Вместе с тем модель уклонилась от единой итоговой метки, рассматривая позитивные и негативные аспекты отдельно, что свидетельствует о смещении фокуса с задачи классификации на задачу выработки рекомендаций.

Self-Refine. Техника обнаружила ограничение, характерное для компактных моделей: на этапе «критики и улучшения» модель неверно интерпретировала задачу и вместо углубления анализа начала переформулировать сам исходный отзыв, дополняя его гипотетическими деталями. Первичный ответ был корректен, однако итеративный этап не только не улучшил, но частично деградировал результат. Это согласуется с наблюдениями о том, что Self-Refine стабильно работает на моделях масштаба от $\sim 7B$ параметров.

Template Pattern. Единственная техника, обеспечившая полностью структурированный и машиночитаемый вывод в точном соответствии с заданным шаблоном. Примечательна самооценка уверенности — «Низкая» — что является корректной оценкой при смешанном характере входного отзыва и указывает на способность модели к метапознанию в рамках заданного шаблона.

Результаты эксперимента позволяют сформулировать несколько ключевых наблюдений.

Влияние формулировки промпта на итоговую метку. При неизменных входных данных пять из шести техник присвоили метку НЕГАТИВНЫЙ, тогда как Zero-Shot — НЕЙТРАЛЬНЫЙ. Это демонстрирует, что способ формулировки запроса является критическим фактором качества классификации даже при использовании одной и той же модели.

Корреляция скорости и детализации. Наблюдается выраженная обратная корреляция между скоростью генерации и объёмом ответа: Few-Shot (4,15 с) против Role Prompting (55,86 с) — разница в 13,5 раза. При этом максимальная

детализация не гарантирует максимальную точность по целевой метрике (классификации).

Зависимость Self-Refine от масштаба модели. Поведение модели gemma3:1b в режиме Self-Refine подтверждает тезис о том, что многоходовые мета-задачи (рефлексия, самооценка) требуют достаточного модельного масштаба. Для стабильной работы данной техники на малых моделях рекомендуется усиление системного промпта с явными ограничениями на область допустимых «улучшений».

Заключение

Проведённый эксперимент показал, что выбор техники Prompt Engineering должен определяться конкретными требованиями целевой задачи и ограничениями системы. Для высокоскоростной классификации в продакшен-системах оптимален Few-Shot Prompting. Для задач, требующих объяснимости и аудируемости решений, предпочтителен Chain-of-Thought. Для генерации структурированного вывода в автоматизированных пайплайнах — Template Pattern. Применение Self-Refine на моделях с числом параметров менее 3B следует ограничить либо дополнить жёсткими инструкциями в системном промпте, исключающими вольную интерпретацию задачи.

Список использованных источников

1. Sahoo, P. et al. *A Systematic Survey of Prompt Engineering in Large Language Models* // arXiv:2402.07927, 2024. — URL: <https://arxiv.org/abs/2402.07927>
2. Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* // Google Research, NeurIPS 2022. — URL: <https://research.google/blog/language-models-perform-reasoning-via-chain-of-thought/>
3. Liu, P. et al. *A Comprehensive Taxonomy of Prompt Engineering Techniques* // arXiv preprint, 2024. — URL: <https://jamesthez.github.io/files/liu-fcs26.pdf>
4. Emergent Mind. *Few-Shot & Chain-of-Thought Prompting: Active Techniques Overview*, 2025. — URL: <https://www.emergentmind.com/topics/few-shot-and-chain-of-thought-prompting>
5. Splunk. *How Chain of Thought (CoT) Prompting Helps LLMs*, 2025. — URL: https://www.splunk.com/en_us/blog/learn/chain-of-thought-cot-prompting.html
6. DAIR.AI. *Prompt Engineering Guide: Chain-of-Thought (CoT) Prompting*. — URL: <https://www.promptingguide.ai/techniques/cot>
7. K2view. *Prompt Engineering Techniques: Top 6 for 2026*, 2026. — URL: <https://www.k2view.com/blog/prompt-engineering-techniques/>
8. Geshan, M. *How to Use Ollama APIs: Generate, Chat and More*, 2025. — URL: <https://geshan.com.np/blog/2025/02/ollama-api/>
9. Ollama. *API Reference: Introduction*. — URL: <https://docs.ollama.com/api/introduction>
10. DAIR.AI. *The Prompt Report: A Systematic Survey of Prompting Techniques*, June 2024. — URL: <https://www.promptingguide.ai/papers>